
UNIDAD 01

Introducción a la informática

Programación en Computación · Ingeniería Electromecánica — 2.º año · UTN FR Reconquista
Longhi Pablo · Talijancic Iván

La materia en tres ideas

Resolver

Traducir un problema de ingeniería a un algoritmo. Eso es lo difícil, no la sintaxis.

Escribir

JavaScript sobre Node.js. Un lenguaje real, con herramientas reales.

Verificar

Probar que el programa hace lo que dijiste. Un programa que no probaste, no funciona.

No vamos a memorizar comandos. Vamos a aprender a **pensar problemas** y a expresarlos de forma que una máquina los ejecute.

ANTES DE ARRANCAR

| Cómo está armada la materia

Dos partes, dos parciales

PARTE 1 • VECTORES

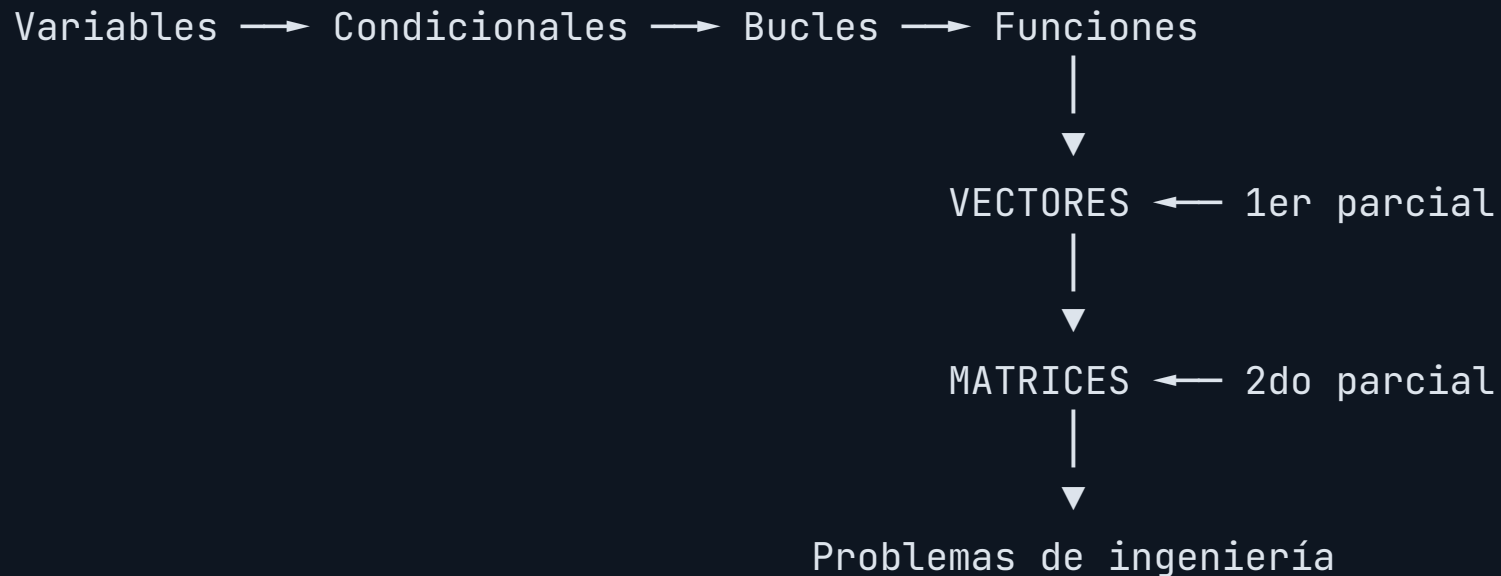
- 01 **Introducción a la informática**
estamos acá
- 02 Entorno de desarrollo y primeros pasos
- 03 Variables, convenciones y operadores
- 04 Condicionales: `if`, `switch`
- 05 Bucles: `for`, `while`, `do-while`
- 06 Funciones
- 07 Arrays unidimensionales (vectores)

PARTE 2 • MATRICES

- 08 Matrices (arrays bidimensionales)
- 09 Operaciones matriciales
transpuesta, simetría, producto
- 10 Integrador: problemas de ingeniería

La parte 2 es más corta en unidades pero más densa: todo lo de la parte 1 se usa adentro de dos bucles anidados.

Todo se apoya en lo anterior



No hay temas independientes. Si te quedás con bucles, vas a arrastrar el problema hasta matrices.

Por eso hay un **TP por unidad**: no se entrega, es para que detectes a tiempo lo que no quedó claro y no acumules deuda hasta el parcial.

Cómo se aprueba

INSTANCIA	CUÁNDO	QUÉ EVALÚA
1er parcial	Al cerrar la parte 1	Vectores: unidades 01 a 07
2do parcial	Al cerrar la parte 2	Matrices: unidades 08 a 10
Recuperatorio	Al final de la cursada	El parcial que quedó pendiente
Final	En las mesas	Un problema integrador

Nota mínima de aprobación: **6 / 10**

EQUIVALE AL 60 % DEL EXAMEN CORRECTO

Los **TP no se entregan ni llevan nota**: son práctica tuya. Pero el parcial se parece mucho a ellos, así que el que los hace llega distinto.

Cómo es un parcial

Un **problema de ingeniería** partido en 5 incisos progresivos. El inciso 1 genera los datos; los siguientes calculan métricas sobre ellos.

Ascensor inteligente
algoritmo del más cercano

Paneles solares
eficiencia por zona

Sensores de planta
alertas por fila

Ejemplos reales de parciales tomados. Son problemas de la carrera, no ejercicios abstractos.

Se rinde **escribiendo código en la computadora**, no en papel. Si resolviste los TP de cada unidad, el parcial no trae sorpresas.

Las reglas de código

Podés usar

for · while · do-while · if · switch · acceso por índice
· .length · .slice()

No podés usar

map · filter · reduce · forEach · find · sort · flat ·
indexOf · splice · retornar objetos

Existen y son útiles. Pero **resuelven por vos justo lo que tenés que aprender**: la lógica del recorrido.

Se corrige también el **código**, no sólo el resultado: funciones con parámetros y retorno, nombres descriptivos y comentarios de cabecera.

Qué vamos a ver

- 01 **Hardware:** las partes físicas y qué hace cada una
- 02 La distinción **RAM vs. almacenamiento**
- 03 **Software:** sistema operativo, drivers y aplicaciones
- 04 Lenguajes **compilados e interpretados**
- 05 Qué pasa cuando escribís `node app.js`
- 06 Sistema **binario:** cómo representa la información

Hoy no escribimos código. Esta clase es el **mapa del terreno**: sin esto, más adelante los errores parecen magia negra.

PARA ARRANCAR

¿Por qué un programa "se cuelga"?

Al final de la clase vas a poder contestar esto, y también por qué al cerrar un programa se pierden los datos.



PARTE 1

| Hardware

La CPU ejecuta las instrucciones

Unidad de Control

Decide qué instrucción se ejecuta y coordina al resto de los componentes.

Unidad Aritmético-Lógica

Hace las cuentas y las comparaciones.

Cuando escribas `if (a > b)`, esa comparación la resuelve **la ALU**.

Cuando escribas `suma = suma + v[i]`, esa suma también.

Y no hace nada más que esto

```
BUSCAR      → traer la próxima instrucción de la memoria
DECODIFICAR → interpretar qué hay que hacer
EJECUTAR    → hacerlo
              y volver a empezar
```

Miles de millones de veces por segundo. Nada más sofisticado que eso.

Por eso un programa se "cuelga"

app.js

```
let i = 0

while (i < 10) {
  console.log(i)
  // falta el i++
}
```

ERROR FRECUENTE

La CPU **no está detenida**: está ejecutando este bucle a toda velocidad, para siempre, porque `i` nunca cambia y la condición nunca se hace falsa. Lo vas a ver en la unidad 05.

La distinción más importante de hoy

	RAM	ALMACENAMIENTO (SSD/HDD)
Volátil	Sí: se borra al apagar	No: persiste
Velocidad	Muy rápida	Lenta en comparación
Capacidad	8 – 32 GB	256 GB – 2 TB
Qué guarda	Los programas mientras corren y sus datos	Archivos y programas instalados

Tus variables viven en la RAM.

CUANDO EL PROGRAMA TERMINA, DESAPARECEN

PREGUNTA

Cargás 31 mediciones y cerrarás el programa. ¿Dónde quedaron?



El resto de las piezas

COMPONENTE	QUÉ HACE
Motherboard	Conecta todo. Los datos viajan por sus <i>buses</i>
GPU	Muchas operaciones simples en paralelo (gráficos)
Fuente	Convierte la tensión de línea en las continuas que usan los componentes
Entrada	Teclado, mouse, sensores : le dan datos a la máquina
Salida	Monitor, impresora, actuadores : muestran o accionan

Un **PLC** o un microcontrolador tiene esta misma estructura —CPU, memoria, entradas y salidas— en un encapsulado chico. Lo que aprendas acá se traslada directo a tu carrera.

PARTE 2

| Software

Capas, y cada una se apoya en la de abajo

Software de aplicación	VSCode, Chrome, TU PROGRAMA
Sistema operativo	Windows, macOS, Linux
Drivers	traducen para cada hardware
Hardware	CPU, RAM, disco, periféricos

El sistema operativo administra

FUNCIÓN	QUÉ RESUELVE
Procesos	Reparte el tiempo de CPU entre los programas abiertos
Memoria	Asigna RAM a cada programa y evita que se pisen
Archivos	Crear, leer, escribir, borrar. Carpetas y permisos
Dispositivos	Coordina la comunicación con los periféricos

Cuando tu programa "no encuentra el archivo", casi siempre es porque le diste una **ruta** que el sistema operativo interpreta distinto de lo que imaginabas. Lo vemos en la unidad 02.

Driver: el traductor

**El mismo Windows funciona
con miles de impresoras.**

No las conoce a ellas: conoce a sus drivers.

Programa $\neq$ proceso

Programa

El **archivo** con las instrucciones.

Vive en el **disco**.

Hay uno.

Proceso

El programa **en ejecución**.

Vive en la **RAM**.

Puede haber varios del mismo programa a la vez.

`app.js` es un programa. `node app.js` crea un **proceso**.

PARTE 3

| Del código a la máquina

PREGUNTA

Vos escribís texto. La CPU entiende ceros y unos. ¿Quién traduce?



Tu código es texto

app.js

```
console.log('Hola mundo')
```

Para la CPU, eso no significa nada. Ella sólo ejecuta instrucciones en **código máquina**: números binarios.

Siempre hay un traductor.

LO QUE CAMBIA ENTRE LENGUAJES ES CUÁNDO TRADUCE

Lenguajes compilados

Se traduce **todo el programa de una vez**, antes de ejecutarlo. El resultado es un archivo ejecutable.

```
codigo.c  →  COMPILADOR  →  programa.exe  →  se ejecuta  
            (una vez)      (codigo maquina)
```

Ejemplos: C · C++ · Rust · Go · Pascal · Fortran

Lenguajes interpretados

No se traduce antes. Un programa llamado **intérprete** lee el código y lo va ejecutando **a medida que corre**.

```
app.js → INTERPRETE → se ejecuta  
      (cada vez que corre)
```

Ejemplos: JavaScript · Python · PHP · Ruby

Las diferencias

	COMPILABO	INTERPRETADO
Cuándo traduce	Una vez, antes de ejecutar	Cada vez que se ejecuta
Velocidad	Más rápido	Más lento
Errores de sintaxis	Aparecen al compilar	Aparecen al ejecutar esa línea
Portabilidad	Hay que recompilar por sistema	El mismo archivo corre en todos
Para probar un cambio	Recompilar y ejecutar	Guardar y ejecutar
Qué se distribuye	El ejecutable	El código fuente

Cuándo conviene cada uno

Compilado

Cuando la **velocidad** manda: sistemas embebidos, control en tiempo real, drivers, motores de simulación.

El PLC de una planta corre código compilado.

Interpretado

Cuando manda la **velocidad de desarrollo**: automatizar cálculos, procesar datos, scripts de ingeniería, web.

Un script que analiza mediciones no necesita microsegundos.

JavaScript es interpretado

Su intérprete es **Node.js**. Por eso lo vas a instalar para la próxima clase: sin él, tu `.js` es sólo un archivo de texto.

Lo bueno

El mismo `app.js` corre igual en Windows, macOS y Linux.
Podés trabajar en la máquina que tengas.

Lo que hay que saber

Los errores aparecen **cuando se ejecuta esa línea**, no antes.
Un error en la línea 50 no se ve hasta llegar ahí.

ERROR FRECUENTE

Java y JavaScript son lenguajes distintos, sin relación entre sí más allá del nombre. Java compila a un formato intermedio y lo ejecuta la JVM.



La realidad es más matizada

Los motores modernos de JavaScript **compilan sobre la marcha** las partes del código que más se repiten, para acelerarlas. Se llama *compilación JIT*.

La división compilado / interpretado sigue siendo útil para entender qué pasa, pero en la práctica los lenguajes combinan las dos estrategias.



PARTE 4

Qué pasa cuando corrés un programa

Te vas a cansar de escribir esto

SALIDA

```
node app.js
```

Vale la pena saber qué ocurre entre que apretás Enter y aparece el resultado.

Paso por paso

- 01 La **terminal** le pide al sistema operativo que ejecute `node`
- 02 El **sistema operativo** lo carga del disco a la RAM y crea un **proceso**
- 03 `node` abre tu `app.js` y lo lee — tu código es un **dato de entrada** para él
- 04 `node` **traduce** tu JavaScript a instrucciones que la CPU entiende
- 05 La **CPU ejecuta**: buscar → decodificar → ejecutar
- 06 Al terminar, el proceso muere y **su memoria se libera**

Lo importante del paso 4

La CPU no entiende JavaScript.

Node.js hace de intérprete: traduce tu código para ella.

Por eso hay que **instalar Node** antes de correr un `.js`. Y por eso el mismo archivo funciona igual en Windows, macOS y Linux: lo que cambia es Node, no tu código. Podés trabajar en la máquina que tengas.

PARTE 5

| Cómo representa la información

Sólo dos estados

Hay tensión o no hay tensión.

Eso es un **bit**: 0 o 1. Todo lo demás se construye a partir de ahí.

Cómo funciona el decimal

Ya sabés esto, aunque no lo pienses así. En el número **2026**, cada posición vale una **potencia de 10**:

2026 EN BASE 10

2	0	2	6
10^3	10^2	10^1	10^0

$$2 \times 1000 + 0 \times 100 + 2 \times 10 + 6 \times 1 = 2026$$

Base **10**: diez dígitos disponibles, del **0** al **9**.

El binario es lo mismo, con dos dígitos

Base 2: sólo 0 y 1. Cada posición vale una **potencia de 2**.

1011 EN BASE 2

1	0	1	1
$2^3 = 8$	$2^2 = 4$	$2^1 = 2$	$2^0 = 1$

$$1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 = 11$$

1011 en binario es 11 en decimal.

MISMO NÚMERO, DOS FORMAS DE ESCRIBIRLO

PREGUNTA

¿Cuánto vale **110** en binario?



Al revés: de decimal a binario

Se divide por 2 repetidamente y se anotan los **restos**. Ejemplo con el **13**:

DIVISIÓN	COCIENTE	RESTO
$13 \div 2$	6	1
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

Los restos se leen **de abajo hacia arriba**: **13** = **1101**

De ahí sale el 256

Un **byte** son 8 bits. Cada bit puede tomar 2 valores, y son independientes entre sí:

$$2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 \times 2 = 2^8 = 256$$

0	0	0	0	0	0	0	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

Con 8 bits se representan los valores de **0 a 255**: 256 combinaciones. Alcanza para todas las letras, los dígitos y los signos de puntuación.

Y de ahí sale el 1024

UNIDAD	EQUIVALE A	POR QUÉ
1 bit	0 o 1	—
1 byte	8 bits	$2^8 = 256$ valores
1 KB	1024 bytes	2^{10}
1 MB	1024 KB	2^{20} bytes
1 GB	1024 MB	2^{30} bytes

No son 1000 sino **1024**: la informática cuenta en potencias de 2, no de 10. Por eso un pendrive de "16 GB" muestra menos capacidad en la computadora.

PREGUNTA

¿Cuánto da $0.1 + 0.2$?



Precisión finita

SALIDA

```
> 0.1 + 0.2  
0.30000000000000004
```

No es un bug del lenguaje. **0.1** no tiene representación exacta en **binario**, igual que **1/3** no la tiene en decimal.

Tiene consecuencias prácticas cuando comparás números con decimales. Lo retomamos en la unidad 03.

Lo que hay que llevarse

CONCEPTO	EN UNA LÍNEA
CPU	Busca, decodifica y ejecuta. Sin parar
RAM	Rápida y volátil . Acá viven tus variables
Disco	Lento y persistente . Acá viven tus archivos
Sistema operativo	Administra procesos, memoria, archivos y dispositivos
Proceso	Un programa ejecutándose, en la RAM
Node.js	El intérprete que ejecuta tu JavaScript

Instalá el entorno

Node.js

Elegí la versión **LTS**, no la "Current".

nodejs.org/en/download

Visual Studio Code

El editor donde vas a escribir todo el año.

code.visualstudio.com

VERIFICÁ QUE QUEDÓ INSTALADO

SALIDA

```
node -v  
v24.19.0
```

ERROR FRECUENTE

Si algo falla, **no te quedes trabado**: anotá el error o sacale una foto y lo resolvemos al principio de la próxima clase. Es normal que tenga vueltas, sobre todo en Windows.

PRÓXIMA CLASE • UNIDAD 02

Tu primer programa

Entorno de desarrollo • Línea de comandos • `console.log()`

Tarea: Node.js LTS + VSCode instalados, y el TP de la unidad 01

Todo el material del curso: github.com/italijancic/pc-2026